DisasterClaw: A georeferenced simulation and agent framework for budgeted UAV disaster inspection

Author names to be confirmed

Affiliations and corresponding-author details to be confirmed,

Abstract

Aerial disaster agents need an environment in which sensing actions can change the available evidence and their task value can be measured. We present DisasterClaw, a georeferenced simulation and agent framework that converts paired disaster imagery into closed-loop single-UAV inspection episodes. The framework combines map-anchored pre- and post-disaster scenes, a fixed-field-of-view observation renderer, simplified vehicle motion, damage perception, language-conditioned task execution, and episode-level instrumentation. We use DisasterClaw to study budgeted reobservation at two levels. Paired predictions for 3,753 buildings in 44 regions quantify both corrections and harmful changes across three camera footprints. At a reobservation fraction of 0.25, calibrated-entropy allocation raises macro-F1 from 0.6118 to 0.6433, compared with 0.6241 for random allocation and 0.6448 for raw entropy. A 160-question closed-loop study finds no statistically distinguishable advantage among the tested online observation rules at near-matched executed cost. An unchanged-view control further shows answer variation in 11 of 160 questions across five Qwen2.5-VL-7B-Instruct generations, concentrated in count and spatial questions. The framework therefore separates changes caused by the observation, the allocation rule, and answer generation. The current evidence is limited to rendered orthographic imagery, simplified kinematics, an exposed perception backbone, and incomplete raw provenance for the new online runs.

Keywords: UAV disaster inspection, Embodied AI, Simulation platform, Active perception, Budgeted reobservation, Vision-language agents

Review status. This manuscript is an author-review draft. Quantitative tables are reconstructed from archived records and the supplied rerun summaries. Author details and the remaining provenance items in Appendix A require confirmation before submission.

1. Introduction

Disaster response creates a demanding setting for aerial embodied intelligence. An inspection agent must interpret a task, place its sensor over a relevant region, decide whether the current evidence is sufficient, and return a usable assessment under limited observation and motion budgets. Static remote-sensing benchmarks measure recognition on fixed images, but they do not expose the decision of where and when to observe again. General aerial navigation environments provide motion and language tasks, yet they do not necessarily connect a changed view to a disaster-specific answer. A reproducible bridge between these settings is needed to study whether an agent's sensing actions improve the decision that motivated them.

The bridge is technically nontrivial. Disaster imagery is often available as georeferenced, orthographic products rather than as a continuously rendered three-dimensional world. Apparent altitude changes must therefore preserve the source-resolution ceiling and maintain a common geographic reference. The agent also operates across several units of analysis: buildings are used by the damage model, regions define task scope, actions incur observation cost, and complete episodes are scored by final answers. Treating these units as interchangeable can make a classifier improvement appear to be an agent improvement. Generation variability adds another ambiguity when a vision-language model can change its answer without any environmental change.

We present DisasterClaw, a georeferenced simulation and agent framework for single-UAV disaster inspection. It transforms paired pre- and post-disaster imagery into an interactive observation environment with a map-anchored scene, a configurable camera footprint, a simplified vehicle state, finite movement and inspection actions, and episode logging. A disaster-response agent receives a natural-language task, a rendered view, and structured spatial evidence; it can search, center on evidence, descend to obtain a narrower footprint, answer, or abstain. The same environment supports an operator-facing console and a headless benchmark runner, so qualitative inspection and quantitative experiments use a shared world and action interface.

DisasterClaw is used here to examine budgeted reobservation. A closer view allocates more source pixels to the target region while discarding some surrounding context. Whether this helps depends on the task: a building label, a count, and a spatial relation can require different evidence. We therefore evaluate the mechanism first on paired building predictions and then in closed-loop question answering. A fixed-view repeated-generation control separates changes caused by a new observation from changes that can arise when the same Qwen input is sampled again. This design makes a weak

or harmful reobservation measurable rather than assuming that another view is beneficial.

The contributions are as follows.

1. We develop a georeferenced aerial disaster-inspection simulation and evaluation framework that combines paired disaster scenes, a camera-footprint model, simplified UAV motion, a finite action interface, an interactive console, and headless episode instrumentation. Its physical and data limits are stated explicitly.
2. We implement a closed-loop disaster-response agent that connects task language, rendered observations, building-damage evidence, spatial state, movement tools, and Qwen2.5-VL-7B-Instruct answer generation. Planning, representation, and memory modules are treated as system components rather than isolated algorithmic contributions.
3. We use the framework for a controlled study of budgeted reobservation. The protocol separates building-level correction and harm, near-matched online action cost, final task correctness, and unchanged-view generation variability. The resulting evidence identifies where improved visual classification does and does not transfer to inspection answers.

The present platform is an imagery-based research environment, not a six-degree-of-freedom flight simulator. It uses orthographic source imagery and simplified motion, and it does not model wind, aircraft dynamics, occlusion, collision avoidance, or battery discharge. These boundaries determine the scope of every experimental claim.

2. Related work

2.1. Aerial embodied agents and simulation platforms

Vision-and-language navigation connects natural-language instructions to sequential visual decisions [1]. AerialVLN extends this problem to UAV navigation, while CityNav introduces large-scale real-world aerial navigation data [2, 3]. OpenFly provides a broader aerial vision-language platform, toolchain, and trajectories across multiple heights and scenes [4]. These works establish language-conditioned aerial motion and height-varying observation as active research areas. DisasterClaw addresses a complementary requirement: constructing task-scoped, paired disaster observations and measuring whether an inspection action changes the requested disaster assessment.

Large pretrained models can be composed with perception and navigation modules without treating the entire stack as a newly learned end-to-end policy [5]. CityNavAgent uses hierarchical semantic planning and global memory, spatial text representations support language-model reasoning in aerial

navigation, and AerialClaw exposes modular aerial skills and feedback [6, 7, 8]. DisasterClaw adopts this modular view. Semantic maps, planning, memory, and tool execution support the implemented agent; the platform contribution lies in their integration with georeferenced disaster scenes, controllable inspection observations, and a common measurement interface.

CityEQA connects active urban exploration to embodied question answering and evaluates answers after view refinement [9]. Its setting is a direct precedent for active aerial question answering. DisasterClaw differs in task domain, source imagery, action abstraction, and its paired damage endpoint. These differences preclude a direct numerical ranking, but they motivate reporting environment construction and task scoring with equal precision.

2.2. Disaster interpretation from remote-sensing imagery

xBD provides paired satellite images and building-damage annotations for disaster assessment [10]. RescueADI studies autonomous tool use for adaptive interpretation of remote-sensing disaster images [11]. Thus, paired damage recognition and agentic disaster analysis both predate this work. DisasterClaw uses such static records as the substrate of an interactive inspection environment: a geographic window becomes an observation, vehicle state changes the window, and the resulting episode is scored at both building and task levels.

This transformation remains constrained by the data. Cropping and resampling an orthographic product can alter the number of pixels assigned to a target, but it cannot synthesize unrecorded viewpoints or detail below the native source scale. The platform is consequently suited to controlled studies of spatial coverage, observation allocation, and answer formation. Evidence from it cannot be presented as a measurement of real UAV image formation or field performance.

2.3. Active perception, uncertainty, and task value

Active perception treats sensing as an action selected in service of a task [12]. Planning under partial observability further connects information gathering with uncertain states and action outcomes [13]. This perspective motivates the central platform requirement: a new observation should be evaluated by the decision it enables and the cost it incurs.

Entropy reduction is a convenient trigger but is not equivalent to expected task utility. A distribution can become more concentrated around an incorrect class, while a view may help a count or spatial relation without changing the most uncertain building label. DisasterClaw therefore records classification, correction and harm, final question correctness, and executed observation count as separate endpoints.

Temperature scaling addresses probability calibration, while evaluation under dataset shift shows why in-distribution calibration should not be treated as a universal reliability property [14, 15]. Selective prediction measures the trade-off between coverage and error, and uncertainty-aware language-model planning has been studied through help-seeking behavior [16, 17]. We use these distinctions to separate calibrated damage probabilities, prediction-set sensing triggers, and the VLM's generated answer. None alone establishes reliability for the composed agent.

3. The DisasterClaw simulation platform

3.1. Scope and system organization

DisasterClaw is an imagery-based simulation and evaluation framework for single-UAV disaster inspection. Its purpose is to turn georeferenced disaster records into closed-loop episodes in which an agent changes its map position or observation footprint, receives a new image, and is scored on a disaster task. The implementation has four connected layers (Fig. 1): a task interface, a disaster-response agent, a simulated geospatial world, and an instrumentation layer. The operator-facing path uses a React/Vite console connected to a Flask/Socket.IO service. Quantitative experiments use a headless runner that calls the same environment and controller modules without rendering the user interface.

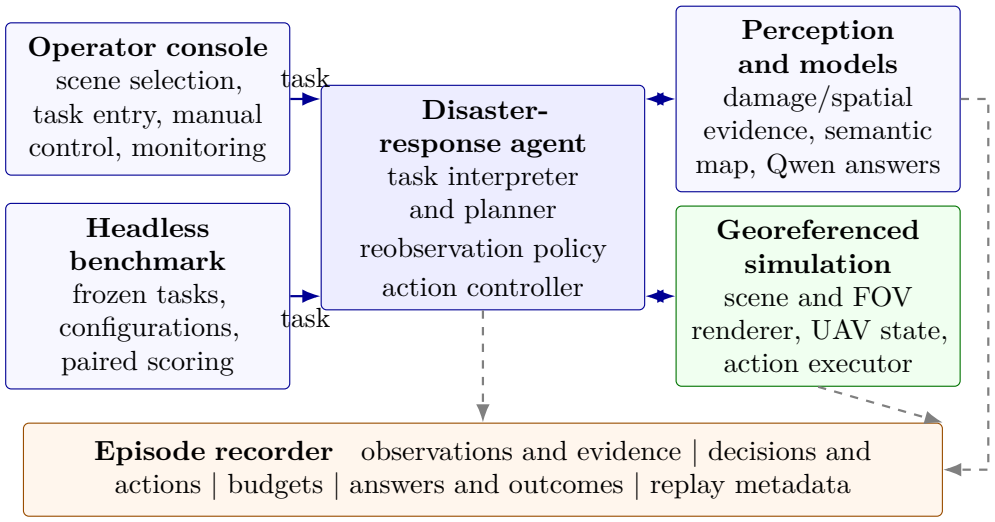


Figure 1: DisasterClaw system organization. The operator console and headless benchmark connect the disaster-response agent to the same perception/model services and georeferenced environment. The recorder retains intermediate model traces, environment transitions, budgets, and task outcomes.

Table 1 states what each layer implements and the boundary attached to it. This distinction is central to the intended use of the platform. Its geospatial state and image renderer support controlled observation studies; they do not reproduce the complete physics of an aircraft or camera.

Table 1: Implemented DisasterClaw capabilities and their evaluation boundaries.

Capability	Implemented function	Boundary in this study
Scene	Georeferenced pre-/post-disaster tiles, building annotations, map anchors, target ROIs, and contextual basemap	Three evaluated events and 44 ROIs; incomplete source archive and no global coverage claim
Observation	Nadir fixed-FOV crop and resampling at three footprint widths; paired archival and current-image channels	Orthographic imagery; no new viewpoint, acquisition time, or detail below the assumed 0.5 m/pixel source scale
Vehicle and actions	Geographic and local position, altitude, heading, speed, task state, straight-line motion, centering, descent, hover, marking, and reporting	One UAV with simplified kinematics; no six-degree-of-freedom dynamics, wind, collision, communication, or battery model
Perception and models	Building localization and four-class damage evidence, spatial summaries, semantic map, planner, and Qwen answer generation	Modular system integration; model revision and some archived runtime fields remain incomplete
Tasks	Operator instructions, damage/presence/count/spatial questions, and archived language-guided navigation instructions	Task-scoped simulation; no claim of unrestricted natural-language or open-world competence
Logging and evaluation	Interactive monitoring, headless episodes, paired task IDs, intermediate evidence, actions, budgets, answers, and terminal scores	New online records are locally represented by supplied summaries; screenshots illustrate function only
Physical boundary	Map-anchored scene transitions and a controlled observation ladder	No photorealistic sensor synthesis, aircraft certification model, hardware-in-the-loop result, or physical-flight validation

3.2. Georeferenced disaster-scene construction

The scene layer reads paired pre- and post-disaster tiles, building polygons, damage labels, and geographic metadata derived from xBD. Pixel-to-geographic and geographic-to-pixel affine transforms place each image on its recorded ground footprint. When a tile becomes active, its center defines the

local world anchor. Robot, target, and report coordinates remain available as latitude, longitude, and altitude, while local north and east offsets are recomputed relative to that anchor.

The interactive map can place the active disaster image over a satellite or street basemap, display the tile boundary, and color annotated buildings for operator reference. These annotation overlays are not passed to the evaluated agent as visual input. In rendered observations, georeferenced disaster tiles supply task evidence and the surrounding basemap supplies spatial context when a wider footprint extends beyond the available disaster tile. Reference answers and building-level scores remain restricted to the annotated target ROI, so contextual background is not treated as disaster ground truth.

The evaluated scene population contains three disaster events and 44 target ROIs. This is the experimental subset, not a claim that the software can reproduce every disaster represented by the source dataset. Missing tiles, sparse geographic coverage, affine-registration error, and differences in acquisition date all constrain the resulting environment.

3.3. Observation renderer and camera ladder

The simulated sensor is nadir-facing, with horizontal field of view θ and fixed output width W. For vehicle height h, the ground-footprint width L and effective ground sampling distance g are

$$L(h) = 2h\tan(\theta/2), \qquad g(h) = \frac{L(h)}{W}. \tag{1}$$

We use $\theta = 60^\circ$, $W = 1024$, and an assumed source scale of 0.5 m/pixel. A 1024-pixel source tile therefore spans 512 m. Footprints of three, two, and one tile widths define cruise, intermediate, and floor states at approximately 1330.2, 886.8, and 443.4 m, with effective GSDs of 1.5, 1.0, and 0.5 m/pixel (Fig. 2). The heights are derived parameters of this camera abstraction; they are not measured flight altitudes from the source imagery.

For paired damage perception, the post-disaster channel is sampled at the current footprint and fixed output size. The corresponding pre-disaster window is retained at the assumed source scale before the pair is brought to the detector input geometry. This represents access to an archival reference and a controllable current observation. It also means the two channels undergo different resampling. The platform records this choice because it can contribute to any observed view response.

Changing height only crops and resamples existing orthographic imagery. The renderer does not create sub-source detail, oblique views, three-dimensional parallax, self-occlusion, atmospheric effects, motion blur, or a new acquisition time. At the floor, further descent cannot add information under the adopted source model.

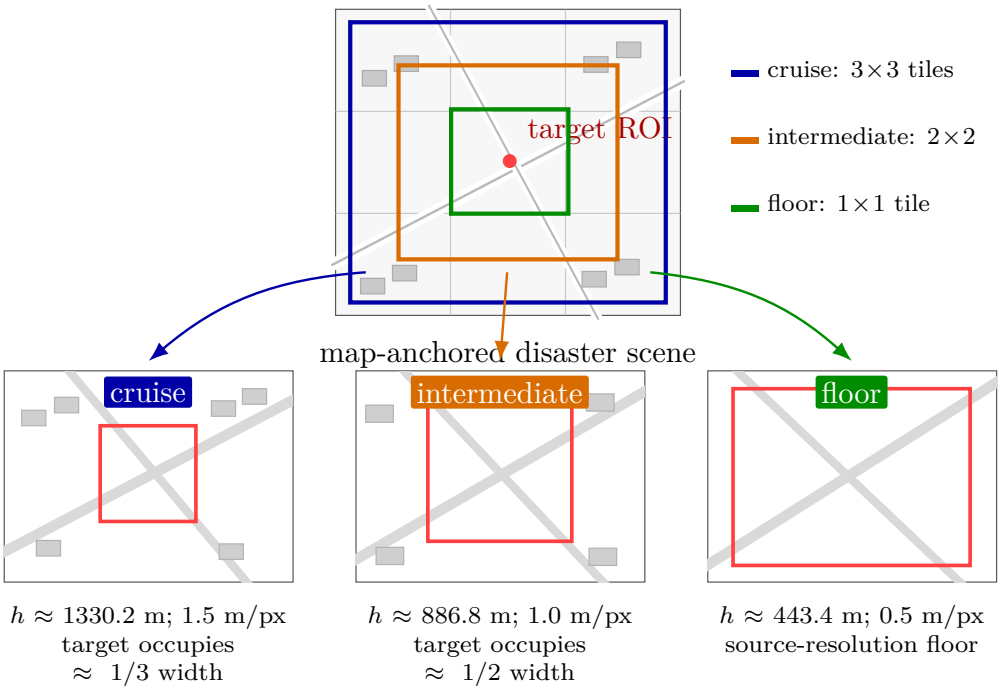


Figure 2: Observation construction from one georeferenced scene. Each state is rendered to 1024×1024 pixels. Descending narrows the ground footprint and assigns more existing source pixels to the target ROI; the floor state reaches, but cannot exceed, the assumed source resolution.

3.4. Vehicle state and action execution

The world model stores a single UAV with geographic position, altitude, heading, speed, flight status, and task state. It also stores targets, reports, the active disaster tile, map bounds, and timestamps. Motion is implemented by straight-line interpolation in a locally anchored horizontal frame. A position update is converted between geographic coordinates and local north–east offsets, and altitude is recorded as the vertical state.

The task planner and closed-loop agent share a finite set of environment tools: fly to a geographic coordinate, move by a relative horizontal or vertical offset, hover, run disaster detection, mark a target, and produce a report. The inspection controller additionally uses answer, abstain, and stop as episode decisions. A reobservation maps to one or more movement primitives, such as centering on evidence and descending by one camera-ladder state, followed by a new render and perception pass.

This action model supports repeatable studies of where the sensor is pointed and how often another observation is requested. It omits rigid-body attitude dynamics, acceleration limits, wind, terrain following, collision checking, communications, and battery discharge. Executed action count is therefore the principal online cost in this paper. Any time estimate obtained from configured speeds remains a simulator quantity rather than measured endurance.

3.5. Interactive operation and experiment instrumentation

The console exposes disaster selection, a georeferenced situation map, manual flight controls, language-task submission, perception displays, agent state, and event logs. An operator can activate a disaster tile, select a target location, issue an inspection task, observe the planned or closed-loop actions, and inspect the returned damage evidence. Figure 4 illustrates these functions after the agent loop has been defined. The screenshots support interface description and diagnosis; they do not supply quantitative evidence.

The headless benchmark initializes a question and vehicle state, invokes the same observation and action modules, and writes the evolving answer, evidence, controller decision, movement, budget, and terminal outcome. Frozen question identifiers allow policies to be compared on paired tasks. The study also supports a fixed-view mode that holds the environment state constant while repeating answer generation. This instrumentation is what permits environmental change and model-generation change to be analyzed separately.

4. Disaster-response agent and reobservation mechanism

4.1. Task modes and closed-loop state

DisasterClaw exposes two task modes over the same environment tools. In plan-and-execute mode, a language planner converts an operator instruction into a finite sequence of typed actions with parameters and reasons; a rule-based fallback handles simple geographic tasks when the planner model is unavailable. In closed-loop inspection mode, the agent alternates observation, evidence extraction, decision, and execution until it answers, abstains, stops, or reaches an episode limit. The experiments in this paper evaluate the latter mode.

An inspection task contains a question q, a target region of interest (ROI), and a reference answer y_q. At step t, the environment state $s_t = (x_t, y_t, h_t)$ identifies the UAV's map position and height. The renderer returns image $I_t = \mathcal{R}(\mathcal{M}, s_t)$ from georeferenced scene $\mathcal{M}$. The agent observation $o_t = (I_t, z_t)$ also includes structured evidence z_t, such as target coordinates, detected-building counts, damage probabilities, and a semantic-map summary. The state notation describes the implemented observation process rather than a complete aircraft dynamics model.

The agent-level decision set is

$$\mathcal{A}_q = \{\text{search}, \text{center}, \text{descend}, \text{answer}, \text{abstain}, \text{stop}\}. \tag{2}$$

Search and center become relative horizontal movements; descend changes to the next state of the camera ladder. Each movement is executed by the platform before a new image and structured state are returned. The task score is assigned only to the terminal answer, while the episode log retains intermediate answers, decisions, and costs.

Figure 3 shows the resulting state machine. The evidence-sufficiency test and the budget/action-feasibility test separate a model decision from an executable environment transition. This distinction also makes the fixed-view control explicit: it repeats answer generation while holding the complete environment observation fixed.

4.2. Perception, spatial evidence, and answer generation

The evaluated damage backend is recorded as `xview2_first`, with a single ResNet-34 configuration and seed 0. It processes pre-/post-disaster image pairs and supplies detected building locations and four-class damage probabilities: no damage, minor damage, major damage, and destroyed. The backend is shared within the principal offline comparisons. Its prior exposure to evaluation events is disclosed in Section 5.

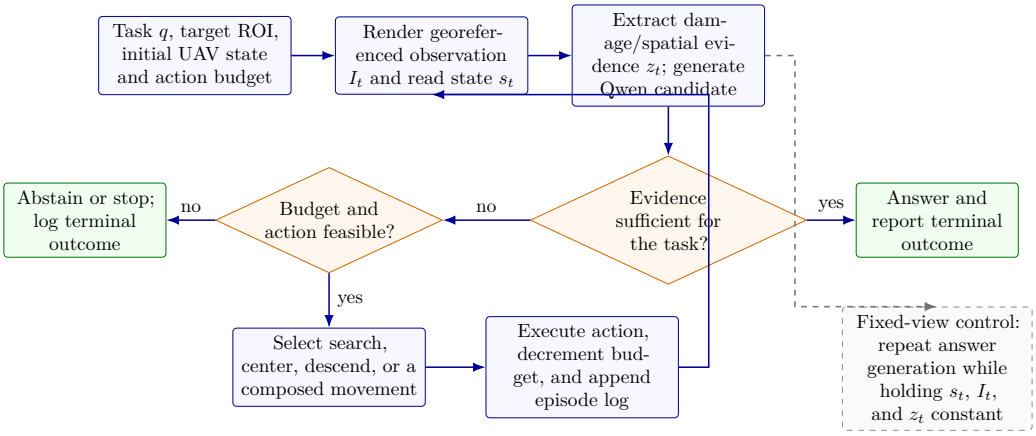


Figure 3: Closed-loop disaster-response agent and reobservation state machine. A reobservation is counted only after a feasible movement is executed and a new georeferenced observation is rendered. The dashed branch denotes the unchanged-view control used to measure answer-generation variation.

Predictions inside the target ROI are matched to annotated buildings by a greedy, one-to-one nearest-centroid rule with a 12 m distance threshold. An unmatched building receives a no-damage probability vector before normalization and temperature scaling. Consequently, the reported building endpoint combines localization, matching, and damage classification. We retain this archived convention, disclose unmatched counts, and add a matched-only sensitivity analysis.

For online inspection, the agent receives the current image, target scope, structured counts, damage evidence, and a semantic-map snapshot. Spatial representations, hierarchical planning, and memory can support grounding and search [6, 7, 5]; in this study they are system components whose individual navigation effects are not isolated. The final structured question-answering call uses the locally loaded Qwen2.5-VL-7B-Instruct model. Its output is normalized against the task's answer vocabulary before scoring.

This modular organization keeps the decision boundary visible. A perception module can change a building label without repairing the requested answer, and Qwen can vary its answer without a new environment observation. The episode record preserves both stages so those cases can be distinguished.

4.3. Uncertainty-guided reobservation

Let r_{ij} denote an uncalibrated probability for damage class j. Temperature scaling produces

$$p_{ij}(T) = \frac{r_{ij}^{1/T}}{\sum_{\ell=1}^{4} r_{i\ell}^{1/T}}, \qquad H(p_i) = -\frac{1}{\log 4} \sum_{j=1}^{4} p_{ij} \log p_{ij}. \tag{3}$$

The implementation normalizes probabilities, clips small values for numerical stability, and rounds normalized entropy to three decimal places for ranking. A positive scalar temperature preserves the highest-probability class but can change uncertainty ordering between buildings.

The online controller evaluates whether to reobserve after producing a candidate answer and associated perception evidence. A trigger can use raw or calibrated entropy, a prediction set containing more than one class, a fixed allocation, or an expected-entropy lookup. For the expected-entropy rule, the controller computes

$$\widetilde{G}_t = \max\{0, H_t - \widehat{\mu}(g_{t+1}, \hat{c}_t)\}, \tag{4}$$

and requests reobservation when $\widetilde{G}_t > 0.02$, subject to the available budget and action checks. Here $\widehat{\mu}$ is a development-set mean entropy indexed by future GSD and predicted damage class. This is a plug-in uncertainty-reduction heuristic, not an estimate of expected question utility given the full task history. Its local damage signal can be misaligned with the evidence needed for a count or spatial question.

An accepted request invokes centering, descent, or both. The next rendered view passes through perception and answer generation, and the controller records the new answer, uncertainty, action, and remaining budget. Valid answers are not rejected solely because their self-reported confidence lies below an answer-confidence threshold; the implemented agent is therefore not described as calibrated selective prediction.

4.4. Offline allocation rules and paired endpoints

The building-level study isolates observation allocation from route construction. Each of N buildings has a cruise probability vector p_i^c and a floor vector p_i^f. For budget fraction b, a policy selects S_b with $k = \text{round}(bN)$ buildings, giving

$$p_i^{(b)} = \begin{cases} p_i^f, & i \in S_b, \\ p_i^c, & i \notin S_b. \end{cases} \tag{5}$$

Each selected building counts as one additional observation in this abstraction. The cost does not represent a route visiting all selected buildings.

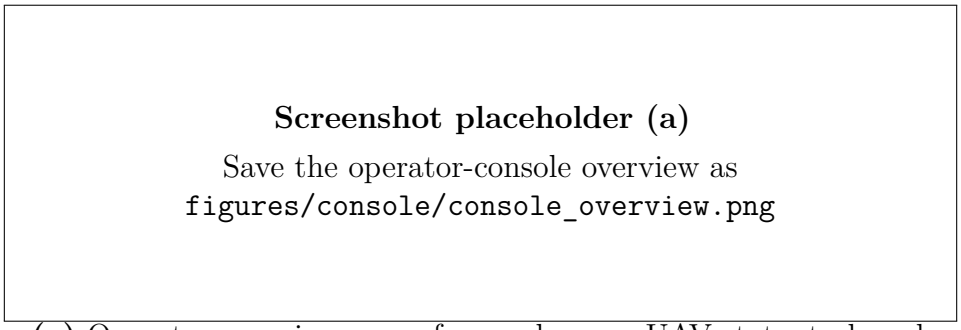
Random allocation samples k buildings without replacement. Raw and calibrated entropy select the largest corresponding cruise entropies. Expected entropy reduction fits the mean paired difference $H(p^c) - H(p^f)$ on development buildings, conditioned on cruise predicted class and one of five equal-width entropy bins. Empty cells fall back to a class mean and then the global mean. The prediction-set rule ranks by the number of classes required to reach a frozen adaptive prediction set threshold [18]. A label-informed diagnostic ranks correctable cases first and is used only for retrospective mechanism interpretation.

Let $\hat{y}_i^v = \arg\max_j p_{ij}^v$. A correctable case has $\hat{y}_i^c \neq y_i$ and $\hat{y}_i^f = y_i$; a harmful case has $\hat{y}_i^c = y_i$ and $\hat{y}_i^f \neq y_i$. For a fixed selected set,

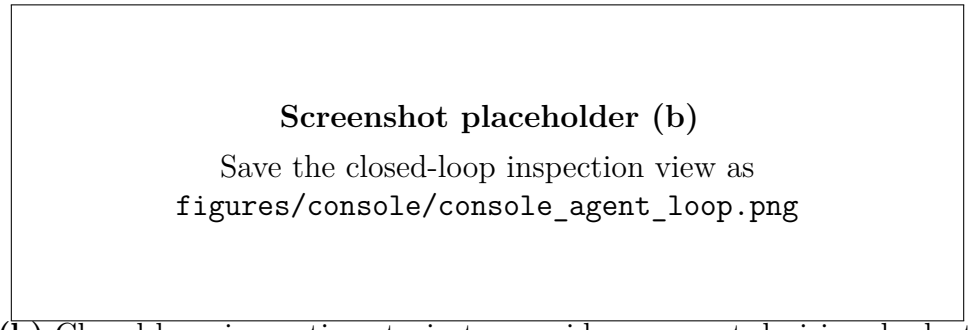
$$\Delta\mathrm{Acc}(S_b) = \frac{N_{\mathrm{corrected}}(S_b) - N_{\mathrm{harmed}}(S_b)}{N}. \tag{6}$$

This identity applies to the hard-label endpoint, not to macro-F1, probability quality, or language-task utility.

Online correctness counts every task in its denominator, including abstentions as unsuccessful answers. Executed reobservation actions are reported separately. If a_q is the terminal answer, task accuracy is $|Q|^{-1} \sum_q \mathbf{1}[a_q = y_q]$. The unchanged-view control holds s_t, I_t, and z_t fixed while repeating the answer-generation call, providing a direct measure of within-input generation variability.



(a) Operator overview: georeferenced scene, UAV state, task and perception panels.



(b) Closed-loop inspection: trajectory, evidence, agent decision, budget and log.

Figure 4: DisasterClaw operator console. Panel (a) summarizes scene selection, the georeferenced map, UAV state, task entry, and perception output. Panel (b) exposes the evolving trajectory, agent decision, evidence, remaining budget, and event log during closed-loop inspection. These screenshots document implemented functions and are not used as quantitative evidence.

5. Experimental protocol

5.1. Evaluation populations and task suite

The study uses three related but non-pooled populations. The building population measures how the platform's camera ladder changes damage predictions under controlled view replacement. The 160-question population measures complete inspection answers and executed observation actions. An archived 200-instruction vision-and-language navigation (VLN) suite diagnoses the integration of planning, recheck, and memory components. Keeping these populations separate prevents building records, question episodes, and navigation instructions from being combined into one sample size.

The development building population contains 2,921 buildings from 20 ROIs in Hurricane Harvey and the Mexico earthquake. The principal evaluation population contains 3,753 unique buildings from 44 ROIs: 1,303 in Hurricane Michael, 1,448 in the Moore tornado, and 1,002 in Nepal flooding. The events used to fit allocation statistics and those used for evaluation are disjoint. The perception checkpoint itself does not satisfy that separation: the frozen manifest marks the `xview2_first` backbone as having prior exposure to evaluation events. Results are interpreted within this boundary.

The closed-loop suite contains 160 unique questions over the same 44 evaluation ROIs, with 58, 47, and 55 questions from the three events. It includes 44 presence, 43 damage, 43 count, and 30 spatial questions. Every policy receives the same question IDs, initial task definitions, and target scopes. The seven-configuration rerun contains 1,120 question–configuration records. The fixed-view control generates five answers for each initial question observation, for 800 outputs. These are repeated evaluations of 160 questions rather than independent tasks.

The archived VLN suite contains 200 instructions from Hurricane Michael, the Moore tornado, Nepal flooding, the Palu tsunami, and the Pinery bushfire. Four switch configurations are compared in one recorded run with seed 42: a keyword/greedy baseline (B0), hierarchical semantic planning (B1), uncertainty recheck (B2), and topological memory (B3). Strict success requires terminating within 25 m of the instruction target. Semantic success uses the broader criterion of terminating within 25 m of a ground-truth building with the requested damage class, and navigation error (NE) is the terminal distance to the strict target. Because this suite used an earlier observation stack and one run per configuration, it is retained as a component diagnostic rather than evidence of algorithmic improvement.

The archived building results permit local arithmetic reconstruction from predictions, labels, and correctness flags. The near-matched online and fixed-view results are currently available in this package as supplied experiment

summaries; their server-side episode files have not been copied into the local audit. The VLN aggregate and episodes are present under `runs/benchmarks/paper_cja_v1/x6_main/` and are transcribed here as author-verified archived results without recomputation. The source image collection, frozen question entities, and exact training data are also incomplete here. Accordingly, the building metrics are independently reconstructed, whereas the new task endpoints receive an internal-consistency audit and the VLN suite retains its archived evaluation scope.

5.2. Platform states, policies, and cost control

The camera states use the geometry in Eq. (1): cruise, intermediate, and floor effective GSDs are 1.5, 1.0, and 0.5 m/pixel. The offline budgets are $b \in \{0, 0.1, 0.25, 0.5, 1\}$, with $b = 0.25$ as the reference operating point. Seven allocation curves are reconstructed: no reobservation, random, raw entropy, calibrated entropy, expected entropy reduction, APS set size, and a label-informed diagnostic.

The primary closed-loop configurations are hold, expected entropy reduction, matched random and fixed allocation, and three motion variants allowing centering, descent, or combined motion. Expected entropy reduction is executed first, and its per-question allocation becomes the reference budget for the dependent controls. Allocated budgets match by question. Expected entropy reduction and center-only motion execute 98 reobservations; the other active controls execute 97; hold executes none. The one-action difference results from execution boundaries such as the altitude floor or early stopping. We therefore call the comparison near-matched in executed cost.

The frozen calibration parameters are $T = 3.5635948726$ and APS threshold $\hat{q} = 0.8064657026$, with nominal $\alpha = 0.1$. Temperature is selected on the development cruise probabilities by minimizing negative log-likelihood over 241 logarithmically spaced candidates in $[0.25, 4]$. The configured online entropy threshold is 0.7 and the expected-reduction threshold is 0.02.

5.3. Model and execution configuration

All vision-language calls reported for the closed-loop answers and archived VLN suite use a locally loaded Qwen2.5-VL-7B-Instruct model, as confirmed by the author. The structured answering interface uses temperature 0.1, a 300-token output limit, and state-level evidence. The current backend defaults to top-$p = 0.9$ and repetition penalty 1.1. These two defaults are not promoted to frozen-run facts because the archived manifests omitted the model fields and environment overrides.

The earlier archived runs use seed 42 and two shards; their manifests record Python 3.11.15, NumPy 2.4.4, PyTorch 2.11, and Transformers 5.5.4.

The exact Qwen revision, prompt hash, effective environment, and dirty source snapshot remain unavailable.

Author query: Archive the Qwen checkpoint revision, effective generation configuration, exact prompt, source snapshot, and new episode-level files with the final submission package.

The fixed-view control uses each question's initial rendered image and structured evidence without changing the UAV state. It invokes the same Qwen answer interface five times and records whether the normalized answer or controller decision changes. This estimates generation variability for a fixed model input; it is not a sensing policy and does not measure variation across model checkpoints.

5.4. Metrics and statistical scope

For the building endpoint, we report accuracy, unweighted macro-F1 over four classes, multiclass Brier score, negative log-likelihood (NLL), and expected calibration error (ECE) with 15 equal-width confidence bins. Brier score is the mean sum of squared differences from the one-hot label without division by class count. Corrected and harmed counts follow Eq. (6). Metrics use the same building population, including unmatched cases under the archived convention.

We add an exploratory paired ROI bootstrap for two comparisons at $b = 0.25$: calibrated entropy versus random allocation and calibrated versus raw entropy. Each of 2,000 resamples draws ROIs with replacement within each observed event, preserving that event's ROI count. Sampled confusion matrices are aggregated before recomputing macro-F1 differences. The seed is 20260905; reported limits are the 2.5th and 97.5th percentiles. Allocation masks, fitted parameters, and the observed event mixture remain fixed. These intervals address sampling variation among ROIs within the three events, rather than training uncertainty, random-policy seeds, or transfer to new disasters.

Closed-loop accuracy counts every question in its denominator. Exact McNemar tests compare paired correctness disagreements on the same 160 question IDs, followed by Holm correction across the reported pairwise tests. Executed observation count is the online cost. For the fixed-view control, an answer flip means that at least two of five normalized answers differ; a decision flip is defined analogously over answer, continue-search, reobserve, and abstain states. The five generations are not treated as independent questions.

The archived VLN table reports Wilson intervals for strict success, semantic success rate (semSR), and mean terminal navigation error. The

configurations were executed once with one seed; the table is descriptive and is not used for pairwise significance claims.

6. Platform-enabled evaluation and results

6.1. Controllable views produce a measurable perception response

The platform generates three observations of the same geographic target scope while varying its footprint and effective GSD. The evaluation retains the same 3,753 building identities across 44 ROIs. Table 2 reports the reconstructed final evaluation population. Moving from cruise to floor increases macro-F1 from 0.6118 to 0.6542 and accuracy from 0.7250 to 0.7423; Brier score and NLL also decrease. The observation control therefore produces a measurable response in the implemented perception pipeline.

Table 2: Building-level results at three simulated camera states on the same 3,753 evaluation buildings. Lower Brier score and NLL are better.

View	GSD (m/px)	Acc.	Macro-F1	Brier	NLL
Cruise	1.50	0.7250	0.6118	0.4814	1.3564
Mid	1.00	0.7357	0.6327	0.4669	1.3314
Floor	0.50	0.7423	0.6542	0.4616	1.3073

Across cruise and floor, 383 buildings change predicted class. Of these, 191 change from incorrect to correct and 126 from correct to incorrect; 66 change between incorrect classes. The net accuracy gain is 65/3,753, or 1.73 percentage points. Thus, a narrower footprint does not act as a uniformly beneficial treatment. It creates both correction opportunities and harmful changes that an allocation rule must distinguish.

Localization contributes to this endpoint. The cruise, intermediate, and floor records contain 912, 916, and 907 unmatched buildings. Among the 191 corrected cases, 35 involve an unmatched record in at least one view; among the 126 harmed cases, 18 do so. On the 2,757 buildings matched at all three views, macro-F1 still rises from 0.7069 to 0.7489 (Table A.1). The response therefore persists in the matched-only subset, although that selected population cannot replace the full endpoint.

These results validate the platform's use for a controlled observation-allocation study. They do not validate the assumed altitude against the original sensor, and they do not show that a physical revisit would reproduce the same change.

6.2. Budgeted allocation on paired observations

At $b = 0.25$, each active rule selects 938 buildings (Table 3). Calibrated entropy obtains macro-F1 0.6433, compared with 0.6241 for random allocation and 0.6118 without reobservation. It produces 53 net additional correct predictions, versus 19 for the recorded random allocation. Uncertainty-based ranking can therefore identify useful reobservations in this observed building population under a common selection budget.

Table 3: Offline allocation at $b = 0.25$. All active rules select 938 floor-view predictions and are scored with the same calibrated vectors. Net correct counts corrections minus harms. The label-informed row uses evaluation labels.

Policy	Actions	Acc.	Macro-F1	ECE	Net correct
No reobservation	0	0.7250	0.6118	0.2423	0
Random	938	0.7301	0.6241	0.2437	19
Raw entropy	938	0.7397	0.6448	0.2578	55
Calibrated entropy	938	0.7391	0.6433	0.2609	53
Expected entropy reduction	938	0.7357	0.6308	0.2551	40
APS set size	938	0.7343	0.6367	0.2549	35
Label-informed diagnostic	938	0.7759	0.6928	0.2853	191

Raw entropy is numerically strongest among deployable rules at this budget, with macro-F1 0.6448. Expected entropy reduction reaches 0.6308 and APS set size reaches 0.6367. The experiment therefore does not establish an advantage for calibrating the ranking score or for replacing uncertainty with the fitted entropy-reduction proxy. The label-informed diagnostic performs better at intermediate budgets by selecting corrections and postponing harmful cases, but it uses evaluation labels and is not deployable.

The exploratory ROI bootstrap gives a calibrated-entropy minus random macro-F1 difference of 0.0192, with interval [0.0097, 0.0419]. The difference from raw entropy is -0.0015, with interval $[-0.0093, 0.0061]$. These estimates support a conditional distinction from the recorded random selection but no clear benefit over raw entropy. They do not quantify transfer to new disasters.

Probability quality follows a different ordering. At $b = 0.25$, calibrated-entropy allocation raises ECE from 0.2423 to 0.2609 and NLL from 1.3564 to 1.3769, although Brier score decreases from 0.4814 to 0.4763. Improved hard-label performance cannot be described as an overall calibration improvement.

6.3. Closed-loop inspection and fixed-view answer variation

The closed-loop suite tests whether building-level view effects transfer to complete inspection answers. Table 4(a) gives the seven paired endpoints at near-matched executed cost. No reported pairwise difference is significant

after Holm correction, and expected entropy reduction does not outperform all matched controls. These results do not support a ranking of the online controllers; their role here is to expose the task-level endpoint of the platform.

The fixed-view control repeats Qwen answer generation five times with the same initial image and structured evidence. Answers vary for 11 of 160 questions (6.9%), while the controller decision varies for one question (0.6%). None of the 43 damage or 44 presence questions changes answer, compared with 4 of 43 count and 7 of 30 spatial questions (Table 4, panel (b)). The other 149 questions retain the same answer across all five generations.

Table 4: Closed-loop and fixed-view controls on the same 160 questions. (a) Allocated budgets match by question; active policies differ by at most one executed action, and no reported pairwise difference is significant after Holm correction. (b) A flip denotes more than one normalized answer across five Qwen2.5-VL-7B-Instruct outputs for the same image and structured evidence.

(a) Closed-loop policy endpoints

Configuration	Correct	Executed actions
Hold	41/160	0
Expected entropy reduction	42/160	98
Random matched	45/160	97
Fixed matched	42/160	97
Center only	40/160	98
Descend only	44/160	97
Combined motion	42/160	97

(b) Fixed-view answer variation by question type

Question type	Questions	Answer flips	Flip rate (%)
Damage	43	0	0.0
Presence	44	0	0.0
Count	43	4	9.3
Spatial	30	7	23.3
All	160	11	6.9

Repeated inference is therefore an unlikely explanation for answer changes on damage and presence questions in this control. Count and spatial questions retain a nonzero generation-variability component, so a changed answer on those task types cannot automatically be attributed to newly acquired visual evidence. The supplied summary reports no case in which repeated questioning converts a consistently wrong answer into a consistently correct one. This observation remains descriptive until the 800 raw generations are archived

locally.

6.4. Archived VLN component diagnostic

The earlier VLN switch experiment provides an additional system-level check (Table A.3). Strict success changes only from 0.070 for the keyword/greedy baseline to 0.080 for hierarchical planning, with overlapping Wilson intervals. Adding uncertainty recheck leaves the reported outcome identical because the recheck mechanism never triggers in B2. Adding topological memory also leaves strict success at 0.080, reduces semantic success from 0.155 to 0.140, and increases mean navigation error from 223.9 to 305.7 m.

This experiment confirms that the planner, recheck, and memory switches were integrated into a complete language-guided navigation path. It does not establish that any of these components improves navigation. In particular, B2 diagnoses an inactive decision path in that archived configuration, and B3 shows that activating memory can alter trajectories without improving strict success. These results support the paper's emphasis on observable component behavior and evaluation boundaries rather than a claimed VLN performance gain.

6.5. What the platform reveals about reobservation

The combined experiments separate three stages that would otherwise be conflated. First, the camera ladder changes the building-level perception endpoint. Second, budgeted ranking can select a more favorable subset of those changes under a common building budget. Third, that advantage does not transfer clearly to the current set of complete inspection answers. The remaining gap lies between the question's evidence requirement, the local damage signal used by the trigger, and Qwen's answer formation.

This diagnosis illustrates the value of the framework beyond a single policy score. DisasterClaw makes the environment state, observation transition, perception response, controller action, and terminal answer separately observable. A subsequent task-conditioned policy can therefore be tested on whether it acquires evidence relevant to damage, count, presence, or spatial reasoning, rather than only on whether a generic uncertainty measure decreases.

7. Limitations and deployment boundary

DisasterClaw constructs observations from existing orthographic products. Its camera ladder changes geographic footprint and resampling density, but it does not synthesize a new viewing angle, time, illumination condition, or detail below the source resolution. Basemap context may come from a

different acquisition date, and the pre- and post-disaster channels undergo different resampling. The assumed flat-ground, nadir geometry is unsuitable for claims about oblique imaging, terrain relief, occlusion, or low-altitude structure recovery.

The vehicle model is deliberately lightweight. Straight-line position updates and configured speeds permit reproducible action sequences, but they omit attitude dynamics, wind, inertia, terrain avoidance, collision risk, communication loss, and battery discharge. Executed action count is therefore an experimental cost surrogate. The platform has not been connected to a flight controller or validated against physical UAV trajectories in this study.

The principal perception backbone has prior exposure to the evaluation events. Development/evaluation separation for allocation statistics does not make the complete perception pipeline event-disjoint. In addition, unmatched buildings receive an archived default class, and only three evaluation events are represented. The results characterize the recorded platform configuration rather than generalization to unseen disasters.

The closed-loop comparison has one execution per configuration, and its active policies differ by one executed action. The fixed-view control measures repeated answer generation but does not replace repeated full-policy runs. The current local package contains summaries of the new online experiments rather than their episode-level records. Although the model family and size are identified as Qwen2.5-VL-7B-Instruct, the exact checkpoint revision, effective generation environment, prompt hash, and dirty source snapshot remain unarchived.

Finally, DisasterClaw integrates planning, spatial representation, memory, perception, and language-model components. The archived VLN switch experiment observes these components in an earlier stack, but its single run, inactive recheck branch, overlapping success intervals, and degraded B3 navigation error do not establish component improvements. These modules should be treated as available system capabilities until they are evaluated repeatedly under the current platform configuration.

8. Conclusion

DisasterClaw turns georeferenced pre- and post-disaster imagery into a closed-loop aerial inspection environment. A shared world model connects a fixed-field-of-view renderer, simplified UAV movement, disaster perception, language-conditioned task execution, interactive monitoring, and headless episode evaluation. This organization makes the consequences of a sensing action observable from the perception output through the final task answer.

The budgeted reobservation study demonstrates this evaluation role. Narrower footprints change the building-level endpoint and create both corrections and harms. Entropy-based allocation can select useful changes under a common building budget, while the tested closed-loop controllers remain statistically indistinguishable on task correctness. Fixed-view repetition further shows that answer generation contributes task-dependent variation, especially for count and spatial questions.

An archived VLN switch experiment further shows why this traceability matters: enabling planning, recheck, and memory does not by itself establish a navigation gain, and an enabled recheck path can remain inactive. The main result is therefore a platform and measurement framework for studying disaster-inspection agents under controlled observation changes. Its next development should add task-conditioned evidence acquisition, repeated policy runs, complete execution archives, and event-disjoint perception checkpoints before extending the claims to unseen disasters or physical flight.

Declarations for author completion

Acknowledgements and funding. *To be supplied by the authors, including funder names, grant numbers, and relevant contributions. No absence of funding is asserted.*

CRediT author statement. *To be assigned and approved by the named authors.*

Declaration of competing interest. *To be confirmed by all authors.*

Data and code availability. The working package contains manuscript source and scripts that reconstruct the reported tables from archived predictions and episode logs. Access to the underlying imagery remains subject to its original terms. A public archive, license, anonymization review, and complete data/checkpoint package have not yet been confirmed; a final availability statement must identify the actual release and its restrictions.

Generative AI assistance. Codex was used to assist manuscript restructuring, English drafting, reference checking, and preparation of analysis and typesetting scripts. The present file is an author-review draft. The responsible authors must verify the final content and supply a disclosure consistent with the journal's current policy; this text does not assert that their verification has already occurred.

Appendix A. Reproducibility and provenance

Appendix A.1. Inputs and reproduction boundary

The manuscript uses a single principal offline population, a fresh two-shard online rerun, an unchanged-view control, and an archived VLN component diagnostic. The offline source directory is `runs/benchmarks/paper_cja_mech_v1/`. Paired evaluation predictions are read explicitly from `final_fov/fov_ladder_eval_items.jsonl`; development pairs come from `fov_ladder_val_items.jsonl`; the reported allocation curves are in `budget_allocation.json`. This explicit selection prevents a missing evaluation filename from silently causing a fallback to development results.

The primary rerun records are `episodes.jsonl` in two shard directories beneath `runs/benchmarks/cja_agent_vqa/`: `paper_cja_mech_final_rerun_shard0of2/` and `paper_cja_mech_final_rerun_shard1of2/`. Their merged reports are in `paper_cja_mech_final_rerun_reports/`. The unchanged-view records are under `cja_en/runs/no_move_reask_shard0/` and `no_move_reask_shard1/`. At the time of this revision, these server-side directories are represented in the local package only by `matched_budget_result.md` and `no_move_reask_result.md`. The main text therefore reports endpoints contained in those summaries and does not claim a local record-level re-audit of the new runs.

The local scripts `cja_en/scripts/audit_downloaded_results.py` and `cja_en/scripts/build_assets.py` read the original archived inputs without modifying them or invoking a model. They regenerate the offline audit and quantitative assets retained from that audit. The audit records input byte counts and full SHA-256 values. The asset generator rejects changed audited inputs and verifies the allocation reconstruction used by the bootstrap. The system and camera-ladder diagrams are schematics of the implemented framework, not synthetic experimental observations.

The paired-building recomputation matches all seven allocation curves at five budgets for accuracy, macro-F1, ECE, Brier score, NLL, executed selections, and net corrections to an absolute tolerance of 10^{-9}. Final three-view metrics also reproduce to that tolerance. These checks establish numerical consistency of available records. They do not certify that the runtime model, original images, or evaluation history can yet be independently recovered.

Appendix A.2. Software correspondence for the platform description

The platform account in Section 3 is tied to the implementation as follows. Scene state, geographic conversion, xBD indexing, and image assembly are implemented in `backend/world.py`, `backend/geo.py`, `backend/xbd_map.py`, `backend/xbd_store.py`, and `backend/mosaic.py`. The camera

abstraction and renderer are implemented in `backend/fov_ladder.py` and `backend/perception.py`. Typed plan execution and the inspection controller are implemented in `backend/ai_planner.py`, `backend/mock_adapter.py`, `backend/agent_vqa.py`, and `backend/recheck.py`; Qwen loading and multimodal calls are contained in `backend/local_qwen_vl.py` and `backend/vlm_analyzer.py`. The operator console is under `frontend/src/`, and the headless protocols used here are under `scripts/benchmarks/`. This map establishes traceability between the manuscript and the supplied software; it is not a claim that every optional module was active in every experiment.

The present archive does not contain a frozen end-to-end platform release with all imagery, basemap cache, checkpoints, environment variables, and interactive replay artifacts. A complete release should add a scene manifest with affine transforms, a dependency lockfile, an executable episode configuration, and one recorded replay whose observations and actions can be regenerated from hashes.

Appendix A.3. Remaining execution requirements

The original frozen manifest was generated on 26 August 2026. Both archived online shard manifests identify source commit `cfb5c827` (abbreviated) and mark the working tree as dirty. The audit retains its full hash. The author confirms that the VLM throughout these experiments was the locally loaded `Qwen/Qwen2.5-VL-7B-Instruct`; the rerun plan records the same model. This resolves the model-family and size question, but not the exact model revision, effective environment overrides, prompt hash, or uncommitted source snapshot.

The frozen question set and dataset entities are needed to verify their manifest hashes, target construction, scoring rules, and relationships between selection and final populations. Checkpoint files and training manifests are needed to characterize perception exposure directly. Although an additional archive identifies a different event-disjoint perception backend, those results are not combined with the principal tables here: changing the backbone changes the experiment, and its checkpoint provenance remains to be verified.

The fresh rerun regenerates reference-dependent controls in new output directories. It binds dependent allocation to the expected-entropy reference by question and reduces the executed-cost difference to one action. Its summary still reports `budget_audit.passed=false`, correctly reflecting that 97 and 98 executed actions are not identical. A submission archive should additionally bind every episode to the reference result hash, source snapshot, effective model configuration, and prompt hash, and should retain requested, allocated, and executed budgets separately.

Appendix A.4. Online controls added after initial review

The near-matched rerun contains seven configurations over the same 160 questions. Expected entropy reduction executes 98 actions; random, fixed, descent-only, and combined-motion controls execute 97; center-only executes 98. The unchanged-view control makes five Qwen calls per initial observation without invoking movement or reobservation. Its aggregate counts are reported in Table 4(b). The two supplied summaries were checked for arithmetic consistency before transcription. Correct-answer totals, action totals, question-type counts, and the expected-entropy versus hold discordances are internally consistent. Some gain/loss fields for the other matched configurations do not equal their difference in correct totals, so those fields are excluded from the manuscript pending the raw `paired_tests.json` records.

Appendix A.5. Sensitivity to unmatched detections

Table A.1 holds the population fixed to buildings with a matched detection at every view. Its 2,757 buildings exclude all cases assigned a default vector in any of the three observations. The view response remains positive for accuracy and macro-F1, while Brier score and NLL decrease. This analysis checks one potential source of the overall response; it is not a representative subset selected independently of the evaluated detector. It also does not repair possible centroid-matching errors or perception training exposure.

Table A.1: Sensitivity analysis on 2,757 buildings matched in all three recorded views. This selected subset does not replace the full 3,753-building endpoint.

View	Accuracy	Macro-F1	Brier	NLL
Cruise	0.7918	0.7069	0.3367	0.6455
Mid	0.8052	0.7321	0.3176	0.6123
Floor	0.8099	0.7489	0.3175	0.6103

Appendix A.6. Exploratory uncertainty estimates

Table A.2 reports the added paired ROI bootstrap. It conditions on fixed fitted scores, allocations, and three observed events. ROIs are the sampling units within events; individual buildings are not treated as independent draws. With only three events and prior backbone exposure, these intervals cannot establish robustness to unseen disasters. No multiple-comparison-adjusted confirmatory claim is made.

Table A.2: Exploratory paired ROI-bootstrap macro-F1 differences at $b = 0.25$, with 2,000 resamples and seed 20260905.

Paired comparison	Macro-F1 difference	95% interval
Calibrated entropy – Random	+0.0192	[+0.0097, +0.0419]
Calibrated entropy – Raw entropy	-0.0015	[-0.0093, +0.0061]

Appendix A.7. Archived VLN switch ablation

The VLN diagnostic is stored under `runs/benchmarks/paper_cja_v1/x6_main/`, which contains `results.json`, `episodes.jsonl`, and the B3 memory record. The manifest names the 200-instruction `vln_recheck_eval_v2.json` suite, hybrid grounding, seed 42, and one repeat. The author confirms that its vision-language model was Qwen2.5-VL-7B-Instruct even though the archived environment block leaves the VLM field blank. Table A.3 transcribes the archived aggregate without recomputation. The earlier observation stack and single run prevent causal or cross-seed claims about the enabled modules.

Table A.3: Archived 200-instruction VLN switch ablation. SR is strict success within 25 m of the specified target; semSR accepts a building of the requested damage class within 25 m. Recheck triggers were zero in B0–B3, including the recheck-enabled configurations.

Config.	Enabled navigation components	n	SR (95% CI)	semSR	NE (m)
B0	Keyword/greedy baseline	200	0.070 [0.042, 0.114]	0.165	222.3
B1	Hierarchical semantic planning	200	0.080 [0.050, 0.126]	0.155	223.9
B2	B1 + uncertainty recheck	200	0.080 [0.050, 0.126]	0.155	223.9
B3	B2 + topological memory	200	0.080 [0.050, 0.126]	0.140	305.7

Appendix A.8. Required extensions, without assumed outcomes

TODO(RELEASE): Freeze a runnable DisasterClaw release and publish a scene-to-episode example with a scene manifest, dependency lock, effective configuration, and action/observation hashes. If physical-flight fidelity is claimed later, evaluate the simplified motion and camera abstractions against an external simulator or flight record.

TODO(EXPERIMENT): Archive the new episode-level records, effective Qwen configuration, prompt hash, and source snapshot. Repeat the complete moving-policy comparison with multiple generation seeds; the unchanged-view control measures answer variability but does not replace repeated policy runs.

TODO(EXPERIMENT): Evaluate a backbone whose training and tuning events exclude the final events, with recoverable checkpoints, image/ROI

manifests, and an explicit rule for unmatched detections. Report matched-only classification alongside an endpoint that penalizes misses, rather than silently changing the original rule.

TODO(EXPERIMENT): Test task-conditioned evidence selection across damage, count, presence, and spatial questions. Record whether the initial answer was wrong, what evidence was acquired, and whether that evidence addressed the unresolved requirement. Compare at matched executed cost, including movement time and inference cost if measured.

TODO(EXPERIMENT): If operational selective answering is claimed, choose thresholds on separate development data and evaluate risk and coverage on held-out tasks. Retrospective confidence plots in this manuscript do not satisfy that design.

References

[1] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, A. van den Hengel, Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 3674–3683.

[2] S. Liu, H. Zhang, Y. Qi, P. Wang, Y. Zhang, Q. Wu, AerialVLN: Vision-and-language navigation for UAVs, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 15384–15394.

[3] J. Lee, T. Miyanishi, S. Kurita, K. Sakamoto, D. Azuma, Y. Matsuo, N. Inoue, CityNav: A large-scale dataset for real-world aerial navigation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025, pp. 5912–5922.

[4] Y. Gao, C. Li, Z. You, J. Liu, Z. Li, P. Chen, Q. Chen, Z. Tang, L. Wang, P. Yang, Y. Tang, Y. Tang, S. Liang, S. Zhu, Z. Xiong, Y. Su, X. Ye, J. Li, Y. Ding, D. Wang, X. Li, Z. Wang, B. Zhao, OpenFly: A comprehensive platform for aerial vision-language navigation, arXiv preprint arXiv:2502.18041v7 (2026). doi:10.48550/arXiv.2502.18041.

[5] D. Shah, B. Osiński, B. Ichter, S. Levine, LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action, in: Proceedings of The 6th Conference on Robot Learning, Vol. 205 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 492–504.

[6] W. Zhang, C. Gao, S. Yu, R. Peng, B. Zhao, Q. Zhang, J. Cui, X. Chen, Y. Li, CityNavAgent: Aerial vision-and-language navigation with hierarchical semantic planning and global memory, in: Proceedings of the 63rd Annual

Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 31292–31309. doi:10.18653/v1/2025.acl-long.1511.

[7] Y. Gao, Z. Wang, P. Han, L. Jing, D. Wang, B. Zhao, Exploring spatial representation to enhance LLM reasoning in aerial vision-language navigation, arXiv preprint arXiv:2410.08500 (2024).

[8] K. Li, J. Yang, L. Zhang, G. Yu, C. Yan, Y. Ding, D. Wang, N. Luo, G. Liu, X. Gao, Q. Wang, AerialClaw: An open-source framework for LLM-driven autonomous aerial agents, arXiv preprint arXiv:2606.12142 (2026). doi:10.48550/arXiv.2606.12142.

[9] Y. Zhao, K. Xu, Z. Zhu, Y. Hu, Z. Zheng, Y. Chen, Y. Ji, C. Gao, Y. Li, J. Huang, CityEQA: A hierarchical LLM agent on embodied question answering benchmark in city space, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2025, pp. 12465–12480. doi:10.18653/v1/2025.emnlp-main.630.

[10] R. Gupta, R. Hosfelt, S. Sajeev, N. Patel, B. Goodman, J. Doshi, E. Heim, H. Choset, M. Gaston, xBD: A dataset for assessing building damage from satellite imagery, arXiv preprint arXiv:1911.09296 (2019). doi:10.48550/arXiv.1911.09296.

[11] Z. Liu, D. Zhao, B. Yuan, RescueADI: Adaptive disaster interpretation in remote sensing images with autonomous agents, arXiv preprint arXiv:2410.13384 (2024). doi:10.48550/arXiv.2410.13384.

[12] R. Bajcsy, Active perception, Proceedings of the IEEE 76 (8) (1988) 996–1005.

[13] A. Curtis, G. Matheos, N. Gothoskar, V. Mansinghka, J. B. Tenenbaum, T. Lozano-Pérez, L. P. Kaelbling, Partially observable task and motion planning with uncertainty and risk awareness, in: Proceedings of Robotics: Science and Systems, 2024. doi:10.15607/RSS.2024.XX.118.

[14] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning, Vol. 70 of Proceedings of Machine Learning Research, 2017, pp. 1321–1330.

[15] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, J. Snoek, Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.

[16] Y. Geifman, R. El-Yaniv, SelectiveNet: A deep neural network with an integrated reject option, in: Proceedings of the 36th International Conference on Machine Learning, Vol. 97 of Proceedings of Machine Learning Research, 2019, pp. 2151–2159.

[17] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, A. Majumdar, Robots that ask for help: Uncertainty alignment for large language model planners, in: Conference on Robot Learning, Vol. 229 of Proceedings of Machine Learning Research, 2023, pp. 661–682.

[18] Y. Romano, M. Sesia, E. J. Candès, Classification with valid and adaptive coverage, in: Advances in Neural Information Processing Systems, Vol. 33, 2020.